

Розділ 11

Регулярні вирази

Дотепер ми зчитували файли, шукали закономірності та витягували різні, потрібні нам фрагменти рядків. Ми використовували рядкові методи, як-от `split` та `find`, а також списки і зрізи рядків для витягування частин рядків тексту.

Оскільки процес пошуку та витягування досить поширений, у Python є потужний модуль *регулярних виразів*, який досить ефективно виконує подібні завдання. Ми не розглядали регулярні вирази раніше через те, що, при всій їхній потужності, вони досить складні, а для опанування їхнього синтаксису потрібно багато часу.

Регулярні вирази – це фактично окрема невелика мова програмування для пошуку та парсингу рядків. На тему регулярних виразів навіть книги пишуть. У цьому розділі ми опрацюємо лише основи регулярних виразів. Детальніше про них можна дізнатися за посиланнями:

https://en.wikipedia.org/wiki/Regular_expression

<https://docs.python.org/library/re.html>

Перед використанням модуля регулярних виразів `re`, його необхідно імпортувати до вашої програми. Найбільш простим прикладом використання модуля регулярних виразів є функція `search()`. Нижченаведена програма демонструє функцію пошуку.

Здійснення пошуку рядків, що містять фрагмент 'From'

```
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('From:', line):
        print(line)
```

Код: <https://www.py4e.com/code3/re01.py>

Відкриваємо файл, проходимо кожним рядком і використовуємо регулярний вираз `search()`, щоб вивести лише ті рядки, що містять «From:». У цій програмі не показано використання регулярних виразів на всю потужність, адже для такого самого результату можна скористатися функцією `line.find()`.

Ефективність регулярних виразів зростає, коли ми додаємо до рядка пошуку спеціальні символи, що дозволяють більш точно визначити, які з рядків збігаються з заданим рядком. Додавання цих спеціальних символів до регулярного виразу дозволяє виконувати складні дії пошуку та витягування за допомогою дуже малої кількості коду.

Наприклад, символ каретки (^) використовується у регулярних виразах для пошуку «початку» рядка. Можна змінити нашу програму так, щоб вона шукала лише ті рядки, у яких на початку знаходиться «From:»:

Здійснення пошуку рядків, що починаються з 'From'

```
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('^From:', line):
        print(line)
```

Код: <https://www.py4e.com/code3/re02.py>

Тепер нам знаходить лише ті ряди, що починаються з «From:». Це все ще досить простий приклад, який можна виконати за допомогою методу **startswith()** з модуля **string**. Але він демонструє, що регулярні вирази містять спеціальні символи дій, які надають більше контролю над пошуком збігу з регулярним виразом.

11.1 Зіставлення символів у регулярних виразах

Існує ряд інших спеціальних символів, які можна використовувати для створення ще більш ефективних регулярних виразів. Найчастіше використовується спеціальний символ – крапка, вона позначає будь-який символ.

У прикладі нижче регулярний вираз **F. .m:** відповідатиме будь-якому з рядків «From:», «Fxxm:», «F12m:», або «F!@m:», адже символ крапки у регулярному виразі позначає будь-які символи.

Здійснення пошуку рядів, що починаються з 'F', за яким стоїть

2 символи, а потім 'm:'

```
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('F. .m:', line):
        print(line)
```

Код: <https://www.py4e.com/code3/re03.py>

11.2. ВИТЯГУВАННЯ ДАНИХ ЗА ДОПОМОГОЮ РЕГУЛЯРНИХ ВИРАЗІВ

Особливо ефективним це є у поєднанні з можливістю вказати у регулярному виразі, що символ може повторюватися необмежену кількість разів за допомогою символів `*` або `+`. Ці спеціальні символи означають, що замість одного символу в рядку пошуку вони відповідають нулю або більше символам (символ «зірочка») або одному або більше символам (символ «плюс»).

Можна ще більше звужити пошукові запити за допомогою повторюваного байдужого символу або символу *узагальнення*, як показано в цьому прикладі:

Здійснення пошуку рядків, що починаються з From і містять символ равлика

```
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('^From: .+@', line):
        print(line)
```

Код: <https://www.py4e.com/code3/re04.py>

Рядок пошуку `^From: .+@` успішно зіставить рядки, що починаються з «**From:** », за яким слідує один або кілька символів (`.` `+`), а потім символ «равлика». Отже, буде знайдено такий рядок:

From: stephen.marquard@uct.ac.za

Можна вважати, що символ узагальнення `.` `+` охоплює всі символи між двокрапкою і знаком «равлика».

From: .+@

Краще сприймати символи «плюс» і «зірочка» як «виштовхувальні». Наприклад, рядок нижче збігатиметься з останнім символом «равлика» у рядку, оскільки символ `.` `+` виштовхує його, як показано тут:

From: stephen.marquard@uct.ac.za, csev@umich.edu, and cwen@iupui.edu

Можна вказати «зірочці» або символу «плюс» не бути такими «жадібними», додавши ще один символ. Про те, як зробити це, дивіться детальну документацію.

11.2 Витягування даних за допомогою регулярних виразів

Щоб витягнути дані з рядка у Python, можна скористатися методом `findall()`, який витягне усі підрядки, що збігаються із регулярним виразом. Розглянемо приклад, у якому необхідно витягнути все, що схоже на адресу електронної пошти, з будь-якого рядка, незалежно від формату. Скористаємося такими даними:

РОЗДІЛ 11. РЕГУЛЯРНІ ВИРАЗИ

```

From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
           for <source@collab.sakaiproject.org>;
Received: (from apache@localhost)
Author: stephen.marquard@uct.ac.za

```

Непотрібно писати код для кожного з типів рядків, розбиваючи і роблячи зріз кожного рядка окремо. Нижче приклад програми, яка використовує функцію `findall()` для пошуку рядків з адресами електронної пошти і виокремлення однієї або декількох адрес з кожного ряду.

```

import re
s = 'A message from csev@umich.edu to cwen@iupui.edu about meeting @2PM'
lst = re.findall('\S+@\S+', s)
print(lst)
# Код: https://www.py4e.com/code3/re05.py

```

Метод `findall()` шукає рядок у другому аргументі та повертає список усіх рядків, які виглядають як адреси електронної пошти. Ми використали послідовність з двох символів, яка збігається з непробільним символом (`\S`).

Результат роботи програми:

```
['csev@umich.edu', 'cwen@iupui.edu']
```

Пояснюємо регулярний вираз: ми шукаємо підрядки, які містять принаймні один непробільний символ, за яким слідує символ «равлика», а за ним ще принаймні один непробільний символ. Символ `\S+` відображає якомога більшу кількість непробільних символів.

Регулярний вираз знайшов два результати (`csev@umich.edu` та `cwen@iupui.edu`), але не відобразив рядок `@2PM`, оскільки *перед* символом «равлика» немає непробільних символів. Цей регулярний вираз можна використовувати у програмі для зчитування всіх рядків у файлі та виведення на екран усіх фрагментів, схожих на адресу електронної пошти, як показано нижче:

```

# Здійснення пошуку рядів, що містять «равлика» між іншими символами
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('\S+@\S+', line)
    if len(x) > 0:
        print(x)
# Код: https://www.py4e.com/code3/re06.py

```

Кожний рядок зчитується, а потім витягуються всі підрядки, що відповідають нашому регулярному виразу.

11.2. ВИТЯГУВАННЯ ДАНИХ ЗА ДОПОМОГОЮ РЕГУЛЯРНИХ ВИРАЗІВ

Оскільки `findall()` повертає список, достатньо перевірити, чи кількість елементів у ньому більше нуля, щоб вивести лише ті рядки, в яких знайдено хоча б один підрядок, схожий на адресу електронної пошти.

Якщо запустити програму із файлом `mbox-short.txt` як аргумента, побачимо такий результат:

```
...
['<source@collab.sakaiproject.org>;']
['<source@collab.sakaiproject.org>;']
['apache@localhost']
['source@collab.sakaiproject.org;']
['cwen@iupui.edu']
['source@collab.sakaiproject.org']
['cwen@iupui.edu']
['cwen@iupui.edu']
['wagnermr@iupui.edu']
```

Деякі з адрес електронної пошти містять некоректні символи, такі як «<» чи «;» на початку або в кінці. Встановимо, що нам потрібна лише та частина рядка, яка починається і закінчується на літеру або цифру.

Для цього скористаємося ще однією особливістю регулярних виразів. Для позначення набору з декількох допустимих символів, які слід врахувати під час пошуку застосовують квадратні дужки. У певному сенсі, `\S` є запитом на пошук кількох символів, «які не є пробілами». Чіткіше вкажемо, які символи слід шукати.

Ось новий регулярний вираз:

```
[a-zA-Z0-9]\S*\S*[a-zA-Z]
```

Тут усе стає трохи складніше, ви можете побачити, чому регулярні вирази – окрема маленька мова. Цей регулярний вираз вказує, що ми шукаємо підрядки, які починаються з *однієї* малої літери, великої літери або цифри «`[a-zA-Z0-9]`», за якою вказано нуль або більше символів, які не є пробілами (`\S*`), після яких стоїть символ «равлика», а йому слідє нуль або більше символів, які не є пробілами (`\S*`), за якими йде велика чи мала літера. Зверніть увагу, ми змінили `+` на `*` для позначення нуля або більше символів, що не є пробілами, оскільки `[a-zA-Z0-9]` уже є одним непробільним символом. Пам'ятайте, `*` або `+` застосовуються до одного символу, що знаходиться безпосередньо зліва від плюса чи зірочки.

З цим виразом у програмі, дані виглядатимуть охайніше:

Здійснення пошуку рядків, у яких між іншими символами міститься символ «равлика»

Символи мають бути літерами або цифрами

```
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('[a-zA-Z0-9]\S*\S*[a-zA-Z]', line)
    if len(x) > 0:
        print(x)
```

Код: <https://www.py4e.com/code3/re07.py>

```
...
['wagnermr@iupui.edu']
['cwen@iupui.edu']
['postmaster@collab.sakaiproject.org']
['200801032122.m03LMFo4005148@nakamura.uits.iupui.edu']
['source@collab.sakaiproject.org']
['source@collab.sakaiproject.org']
['source@collab.sakaiproject.org']
['apache@localhost']
```

Зауважте, у рядках **source@collab.sakaiproject.org** наш регулярний вираз видалив два символи в кінці рядка («>»»). Це сталося тому, що коли ми додали **[a-zA-Z]** в кінець нашого регулярного виразу, ми вказали, що будь-який рядок, який знайде парсер регулярних виразів, повинен закінчуватися на літеру. Отже, коли він виявляє «>» в кінці **«sakaiproject.org>»**, то просто зупиняється на останній знайдений «відповідний» літері (тобто, «g» була останньою відповідністю).

Також врахуйте, що результатом програми є список Python, єдиним елементом якого є рядок.

11.3 Поєднання пошуку та витягування

Іноді необхідно знайти числа в рядках, які починаються з «X-», наприклад:

```
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000
```

Нас не цікавлять будь-які числа з рухомою крапкою з будь-яких рядків, потрібні лише числа з рядків, які мають вказаний вище синтаксис.

Для відбору рядків можна побудувати такий регулярний вираз:

```
^X-.*: [0-9.]+
```

Він означає, що нам потрібні рядки, які починаються з **X-**, за яким йде нуль або більше символів (**.***), потім двокрапка (**:**), а далі пробіл. Після пробілу потрібен один або більше символів, які є або цифрами (**0-9**), або крапкою **[0-9.]**+. Врахуйте, що всередині квадратних дужок крапка відповідає дійсній крапці (тобто, це не символ узагальнення між квадратними дужками).

Це дуже стислий вираз, який майже повністю зіставлятиме лише потрібні нам рядки, як показано нижче:

```
# Здійснення пошуку рядків, що починаються з 'X', за яким слідує символи,
# які не є пробілами та ':'
# після яких стоять пробіл та довільне число.
# Число може містити десяткові знаки.
import re
hand = open('mbox-short.txt')
for line in hand:
```

11.3. ПОЄДНАННЯ ПОШУКУ ТА ВИТЯГУВАННЯ

```
line = line.rstrip()
if re.search('^X\S*: [0-9.]+', line):
    print(line)
```

Код: <https://www.py4e.com/code3/re10.py>

Після запуску програми бачимо чудово відфільтровані дані, які відображають лише потрібні нам рядки.

```
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000
X-DSPAM-Confidence: 0.6178
X-DSPAM-Probability: 0.0000
...
```

Однак тепер потрібно вирішити задачу витягування чисел. Незважаючи на те, що можна просто використати `split`, існує ще одна особливість регулярних виразів, яка дозволяє водночас шукати і парсити рядок.

Ще один спеціальний символ регулярних виразів – круглі дужки. У регулярних виразах, дужки ігноруються під час пошуку рядка, але під час застосування `findall()` вони вказують на те, що вам потрібен не весь вираз, а лише та частина підрядка, яка збігається з регулярним виразом.

Тож вносимо такі зміни до нашої програми:

```
# Здійснення пошуку рядків, що починаються з 'X', за яким йдуть
# символи, які не є пробілами, та '.', після яких йде пробіл
# та будь-яке число. Число може містити десяткові знаки.
# Далі виведення числа, якщо воно більше нуля.
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('^X\S*: ([0-9.]+)', line)
    if len(x) > 0:
        print(x)
```

Код: <https://www.py4e.com/code3/re11.py>

Замість виклику `search()`, додаємо круглі дужки навколо частини регулярного виразу, яка відображає число з рухомою крапкою, щоб вказати, що функція `findall()` має повернути лише частину знайденого рядка, яка містить число з рухомою крапкою.

Результат роботи програми:

```
['0.8475']
['0.0000']
['0.6178']
```

```
['0.0000']
['0.6961']
['0.0000']
...
```

Числа все ще знаходяться в списку та їх потрібно перетворити з рядків на числа з рухомою крапкою, однак ми використали можливості регулярних виразів як для пошуку, так і для витягування потрібної нам інформації.

Наведемо ще один приклад цієї техніки. У файлі деякі рядки мають такий вигляд:

Details: <http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772>

Якщо було б необхідно витягнути всі контрольні числа (ціле число в кінці цих рядків), використовуючи ту саму техніку, що і вище, то можна було б написати таку програму:

```
# Здійснення пошуку рядків, що починаються з 'Details: rev='
# після чого ідуть числа
# Потім виведення числа, якщо хоч одне знайдено
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('^Details:.*rev=([0-9]+)', line)
    if len(x) > 0:
        print(x)
# Код: https://www.py4e.com/code3/re12.py
```

Отже, ми шукаємо рядки, які починаються з **Details:**, за яким йде довільна кількість символів, потім **rev=**, а після цього одна або більше цифр. Потрібно знайти рядки, які відповідають усьому виразу, але слід витягнути лише ціле число в кінці рядка, тож беремо **[0-9]+** у круглі дужки.

Після запуску програми маємо такий результат:

```
['39772']
['39771']
['39770']
['39769']
...
```

Пам'ятайте, **[0-9]+** – «жадібний» символ, який намагається створити якомога довший рядок цифр перед тим, як витягнути їх. Через таку «жадібність» ми отримуємо всі п'ять цифр для кожного числа. Модуль регулярного виразу рухається в обох напрямках до тих пір, поки не натрапить на символ, який не є цифрою, або на початок чи кінець рядка.

Тепер можемо скористатися регулярними виразами, щоб переробити вправу з попереднього розділу, де ми були зосереджені на часі доби кожного повідомлення електронної пошти. Шукали такі рядки:

11.3. ПОЄДНАННЯ ПОШУКУ ТА ВИТЯГУВАННЯ

From stephen.marquard@uct.ac.za Sat Jan 5 09:14:16 2008

і намагалися витягти годину дня для кожного рядка. Ми робили це за допомогою двох викликів `split`. Спочатку розбивали рядок на слова, а потім витягували п'яте слово і знову розбивали його за двокрапкою, щоб витягнути два потрібні нам символи.

Хоч це і спрацювало, код вийшов досить крихким, оскільки ми припустили, що рядки правильно відформатовані. Якщо додати більше перевірок помилок (або додати великий блок `try/except`), щоб запобігти збоям у роботі програми під час неправильного форматування рядків, код збільшиться до 10-15 рядків, які важко буде зрозуміти.

Значно простіше зробити це за допомогою цього регулярного виразу:

```
^From .* [0-9] [0-9] :
```

Цей вираз шукає рядки, які починаються з **From** (зверніть увагу на пробіл), далі йде довільна кількість символів (`.*`), пробіл, дві цифри `[0-9] [0-9]` і в кінці двокрапка. Таким чином, визначаємо, які саме рядки ми шукаємо.

Для того, щоб витягнути лише годину за допомогою `findall()`, додаємо круглі дужки навколо двох цифр ось так:

```
^From .* ([0-9] [0-9]) :
```

У результаті маємо таку програму:

```
# Здійснюємо пошук рядків, що починаються з From та символом,
# після нього йде двозначне число від 00 до 99, за яким йде ':'
# Потім виведення числа, якщо хоч одне знайдено
import re
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    x = re.findall('^From .* ([0-9] [0-9]):', line)
    if len(x) > 0:
        print(x)
# Код: https://www.py4e.com/code3/re13.py
```

Після запуску програма видає наступне:

```
['09']
['18']
['16']
['15']
...
```

11.4 Символ екранування

Позаяк у регулярних виразах використовуються спеціальні символи для позначення початку та кінця рядка або символу узагальнення, потрібно певним чином зазначати, що деякі символи «звичайні» і вони мають відповідати дійсному символу, наприклад, знаку долара або каретці.

Щоб вказати, що потрібно просто знайти певний символ, додайте до нього обернену скісну риску (\). Наприклад, суму грошей можна знайти за допомогою такого регулярного виразу.

```
import re
x = 'Ми щойно отримали $10.00 за печиво.'
y = re.findall('\$[0-9.]+' , x)
```

Оскільки перед символом долара стоїть обернена скісна риска, він відповідає знаку долара у рядку вводу, а не «кінцю рядка», а решта регулярного виразу відображає одну або кілька цифр чи символ крапка. *Важливо:* Символи у квадратних дужках не є «спеціальними». Тобто, [0-9.] означає цифри або крапку. Поза квадратними дужками крапка – це символ узагальнення, що відповідає будь-якому символу. Всередині квадратних дужок крапка – це крапка.

11.5 Підсумки

Попри те, що ми лише поверхнево ознайомилися з регулярними виразами, ми дізналися дещо про мову регулярних виразів. Це пошукові рядки зі спеціальними символами, які вказують системі регулярних виразів на те, що визначатиме «збіг» і що витягувати зі знайдених рядків. Нижче описано деякі зі спеціальних символів і послідовностей символів:

^ Відповідає початку рядка.

\$ Відповідає кінцю рядка.

. Відповідає будь-якому символу (символ узгодження).

\s Знаходить символ, який є пробілом.

\S Знаходить символ, який не є пробілом (протилежне \s).

* Застосовується до найближчого(их) попереднього(их) символу(ів) і вказує на збіг з нулем або більшою кількістю повторень.

*? Застосовується до найближчого(их) попереднього(их) символу(ів) і вказує на збіг з нулем або більшою кількістю повторень у «нежадібний (ледачий) спосіб».

+ Застосовується до найближчого(их) попереднього(их) символу(ів) і вказує на збіг з одним або більшою кількістю повторень.

+? Застосовується до найближчого(их) попереднього(их) символу(ів) і вказує на збіг з одним або більшою кількістю повторень у «нежадібний (ледачий) спосіб».

11.6. КОРИСНЕ ДЛЯ КОРИСТУВАЧІВ UNIX / LINUX

? Застосовується до найближчого(их) попереднього(их) символу(ів) і вказує на збіг з нулем або одним повторенням.

?? Застосовується до найближчого(их) попереднього(их) символу(ів) і вказує на збіг з нулем або одним повторенням у «нежадібний (ледачий) спосіб».

[**aeiou**] Знаходить збіг одного, вказаного у переліку символу. У цьому прикладі це буде «а», «е», «і», «о», або «и», але не жоден інший символ.

[**a-z0-9**] За допомогою знаку мінус можна конкретизувати і вказати діапазон символів. У наведеному прикладі це має бути один символ, а саме мала літера або цифра.

[**^A-Za-z**] Каретка перед переліком позначає логічну інверсію. Тобто, цей приклад вказує знайти збіг з будь-яким символом, *окрім* великої чи малої літери.

() У регулярних виразах під час пошуку дужки ігноруються, але в разі використання **findall()** вони дозволяють витягнути певну частину знайденого рядка, а не весь рядок повністю.

\b Знаходить порожній рядок, однак лише на початку чи в кінці слова.

\B Знаходить порожній рядок, однак не на початку чи в кінці слова.

\d Знаходить будь-які десяткові цифри; аналог переліку [0-9].

\D Знаходить будь-який символ, який не є цифрою; аналог переліку [^0-9].

11.6 Корисне для користувачів Unix / Linux

Ще в 1960-х роках в операційну систему Unix була вбудована можливість пошуку файлів за допомогою регулярних виразів. Тепер вона доступна майже в усіх мовах програмування у різному вигляді.

В Unix вбудовано утиліту інтерфейсу командного рядка *grep* (англ. Generalized Regular Expression Parser – парсер загальних регулярних виразів), яка робить майже те саме, що й наведені у цій главі приклади **search()**. Тому, якщо ви маєте систему Macintosh або Linux, можете спробувати наведені команди у вікні командного рядка.

```
$ grep '^From:' mbox-short.txt
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
```

Тут **grep** вказали відобразити рядки, що починаються з рядка «From:» у файлі *mbox-short.txt*. Поекспериментувавши з командою **grep** та розглянувши її документацію, можна виявити незначні відмінності між підтримкою регулярних виразів у Python і підтримкою регулярних виразів у **grep**. Наприклад, у **grep** не передбачено підтримку символу, який не є пробілом **\S**, доводиться використовувати дещо складніше позначення – [**^**], яке просто здійснює пошук будь-якого символу, який не є пробілом.

11.7 Налаштування програми

У Python вбудована базова документація, яка допомагає швидко пригадати точну назву певного методу. Її можна переглянути в інтерпретаторі Python в інтерактивному режимі.

Щоб відкрити інтерактивну довідкову систему введіть `help()`.

```
>>> help()
help> modules
```

Якщо ви знаєте, який модуль вам необхідно використати, можете скористатися командою `dir()`, щоб знайти методи в модулі, як показано нижче:

```
>>> import re
>>> dir(re)
[.. 'compile', 'copy_reg', 'error', 'escape', 'findall',
'finditer', 'match', 'purge', 'search', 'split', 'sre_compile',
'sre_parse', 'sub', 'subn', 'sys', 'template']
```

Також можна переглянути коротку документацію до певного методу, скориставшись командою `help` разом з потрібним методом.

```
>>> help (re.search)
Help on function search in module re:
```

```
search(pattern, string, flags=0)
    Scan through string looking for a match to the pattern, returning
    a match object, or None if no match was found.
>>>
```

Вбудована документація не надто детальна, але вона стане в пригоді, якщо ви поспішаєте або не маєте доступу до браузеру чи пошукової системи.

11.8 Словник

brittle code (крихкий код) Код, який працює за умови, що вхідні дані мають певний формат; у разі відхилення від потрібного формату, схильний до збоїв. Код називають «крихким», бо він легко призводить до вилетів.

greedy matching (жадібний пошук) У регулярних виразах, це спосіб, де символи `+` та `*` здійснюють пошук навсібіч, щоб знайти максимально можливу кількість символів у рядку.

grep Команда, доступна у більшості Unix-систем, яка шукає у текстових файлах рядки, що збігаються з регулярними виразами. Назва команди розшифровується як «Generalized Regular Expression Parser» (укр. парсер загальних регулярних виразів).

11.9. ВПРАВИ

regular expression (регулярний вираз) Мова для більш складних пошукових рядків. Регулярний вираз може містити спеціальні символи, які вказують на пошук збігів лише на початку або в кінці рядка, або інші подібні параметри.

wild card (символ узагальнення) Спеціальний символ, який відповідає будь-якому символу. У регулярних виразах символом узагальнення є крапка.

11.9 Вправи

Вправа 1: Напишіть просту програму, яка імітує роботу команди **grep** в Unix. Виведіть на екран запит користувачеві на введення регулярного виразу та підрахуйте кількість рядків, що відповідають регулярному виразу:

```
$ python grep.py
```

Введіть регулярний вираз: ^Author

У файлі mbox.txt знайдено 1798 рядів, що мають збіг з ^Author

```
$ python grep.py
```

Введіть регулярний вираз: ^X-

У файлі mbox.txt знайдено 14368 1798 рядів, що мають збіг з ^X-

```
$ python grep.py
```

Введіть регулярний вираз: java\$

У файлі mbox.txt знайдено 4175 рядів, що мають збіг з java\$

Вправа 2: Напишіть програму для пошуку рядків заданого вигляду:

```
New Revision: 39772
```

Витягніть число з кожного рядка за допомогою регулярного виразу та методу **findall()**. Обчисліть середнє арифметичне чисел і виведіть його як ціле число (integer).

```
Enter file:mbox.txt
```

```
38549
```

```
Enter file:mbox-short.txt
```

```
39756
```

